StratasLocal: Portfolio Project Guide

A reproducible, local-first DevOps platform built from the KodeKloud lab skills. Docker, minikube, Linux and Git only, with a clean path to AWS later.

1. Purpose and positioning

The KodeKloud labs proved you can *execute* tasks: create users, harden SSH, run containers, roll back deployments, manage AWS resources. They are imperative, one-off and graded by a checker. Employers and clients want to see the next level: the same skills turned into **declarative, automated, tested, documented and version-controlled** systems.

StratasLocal rebuilds a classic multi-tier datacenter environment (app servers, a database server, a jump host) as code that anyone can clone and run with one command. Every lab skill becomes a component of the project, and every gap becomes a new phase.

Portfolio pitch (one line): *"I took hands-on sysadmin, Docker and Kubernetes lab work and rebuilt it as a version-controlled, CI-tested, locally reproducible platform, designed to swap in AWS with minimal change."*

2. What the labs already give you

Lab area	Skills proven	Where it lands in the project
Linux admin	Users/groups, UIDs, shells, expiry, permissions, ACLs, cron, sshd hardening, PAM limits, SELinux, firewalld, timezone, archives, `sed`, package install	Phase 1 (hardened base images and verification scripts)
Docker	Containers, images, networks, volumes, port mapping, `docker cp`	Phase 2 (multi-container app, Compose, secure Dockerfiles)
Kubernetes	Pods with shared `emptyDir`, ReplicaSets, rolling updates, rollbacks, Jobs, CronJobs	Phase 3 (full app on minikube)

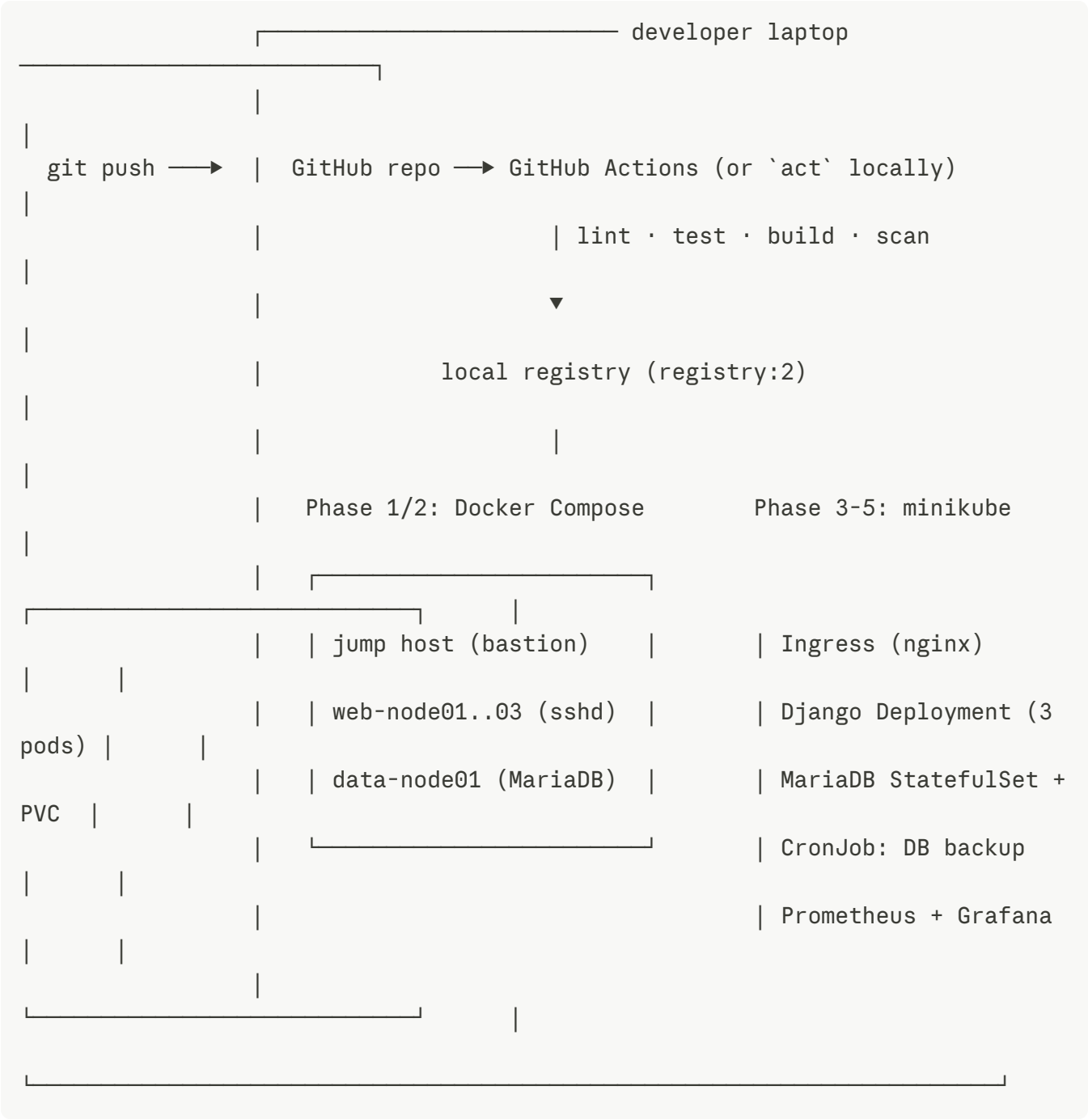
Lab area	Skills proven	Where it lands in the project
AWS CLI	Key pairs, security groups, subnets, EC2, EBS, snapshots, IAM, Elastic IPs, S3 sync	Phase 8 (deferred integration; designed for now)
Troubleshooting	`bind` vs `named`, `nologin` paths, MariaDB datadir relocation, strict grader string checks, dropped sudo	Runbooks and ADRs (Phase 7)

3. Gaps this project closes

The labs were task-driven, so these were never exercised:

1. **Declarative and idempotent automation.** Labs were manual commands; nothing could be re-run safely.
2. **Git workflow.** No branching, review flow, tagging or release practice.
3. **CI/CD.** No pipeline that builds, tests, scans and deploys.
4. **Kubernetes beyond the basics.** No Services/Ingress, ConfigMaps/Secrets, PersistentVolumes, probes, resource limits, RBAC, Helm or Kustomize.
5. **Infrastructure as Code.** AWS was done through CLI only, with no Terraform or Ansible.
6. **Observability.** No metrics, logs or alerts.
7. **Security scanning and supply chain.** No image scanning, linting or non-root enforcement.
8. **Testing and verification.** Two lab failures (the `chmod a+x` task and the EC2 SSH task that expired) show the value of automated checks and scripted, repeatable flows.
9. **Documentation.** No README, architecture diagram, runbook or decision records.

4. Target architecture



The app can be a small Django API with a React or plain HTML front end, since that is already your stack. Keep it deliberately small so the infrastructure is the star.

5. Repository structure

```
strataslocal/
├── README.md                 # pitch, architecture diagram, quickstart, demo
GIF
├── Makefile                  # make up | down | test | lint | k8s-up | scan
├── docs/
|   ├── architecture.md       # mermaid diagrams
|   ├── adr/                  # architecture decision records
|   ├── runbooks/             # incident-style how-tos from lab gotchas
|   └── lab-mapping.md        # KodeKloud lab -> project component
├── app/                      # Django app + Dockerfile
├── compose/                  # docker-compose.yml, per-server images
├── linux/                    # hardening scripts + verification tests
├── k8s/
|   ├── base/                 # Kustomize base manifests
|   └── overlays/{local,aws}/ # aws overlay stays a stub until later
├── charts/strataslocal/      # Helm chart (Phase 5)
├── ci/                       # workflow files, hooks
├── iac/
|   ├── modules/              # provider-agnostic interfaces
|   └── envs/{local,aws}/     # aws env stubbed for future
└── .github/workflows/
```

6. Phases

Each phase has: **goal**, **build**, **definition of done (DoD)** and **portfolio evidence**. Commit at least one meaningful change per phase on its own branch.

Phase 0: Git foundation (do this first)

Goal: professional version-control habits from commit one.

Build

- Initialise the repo, add `.gitignore`, `LICENSE`, and a README skeleton.
- Adopt **trunk-based development with short-lived feature branches** (`feat/linux-hardening`, `fix/mariadb-datadir`).
- Use **Conventional Commits** (`feat:`, `fix:`, `docs:`, `chore:`) and enforce with a `commit-msg` hook or `commitlint`.

- Add `pre-commit` with `shellcheck`, `hadolint`, `yamllint`, and a secrets scanner (`gitleaks`).
- Practise deliberately: `git rebase -i` to tidy a branch, `git cherry-pick` a hotfix, `git bisect` to find a planted bug, `git stash`, `git reflog` to recover a "lost" commit, resolving a real merge conflict, and annotated tags (`v0.1.0`) with a changelog.
- Open PRs even though you are the only contributor, using a PR template with a checklist. Squash-merge.

DoD: protected-looking history (linear, readable), a tagged release, and a `CONTRIBUTING.md` describing the flow.

Evidence: clean commit graph screenshot, PR examples, release notes.

Phase 1: Linux baseline as code (from the Linux labs)

Goal: turn every sysadmin lab into a repeatable, testable script.

Build

- Create Dockerfiles for `web-node` and `data-node` images on a Rocky/AlmaLinux base (matches your RHEL/RH124 foundation), with `sshd` running. Compose them with a `bastion` jump host that can SSH to each.
- Write **idempotent** scripts (`linux/harden.sh`), safe to run twice:
  - Users, groups, UIDs, custom shells (check `/usr/sbin/nologin` vs `/sbin/nologin` first), account expiry, home directories.
  - Permissions and ACLs (`setfacl`), including a script that mimics the `chmod a+x` lab and verifies the result.
  - `sshd_config` hardening: `PermitRootLogin no`, key-only auth; validate with `sshd -t` before restart.
  - Cron with `cron.allow` and `cron.deny`, plus a scheduled backup entry.
  - PAM limits in `/etc/security/limits.conf` (confirm the PAM module is enabled).
  - Package management: install `bind` / `named`, `cronie`, `traceroute`, and document the `bind` vs `named` gotcha.
  - Timezone, archive creation and extraction, `sed -i` config edits.
- Write **verification tests** (`bats-core` or plain shell asserts) that behave like the KodeKloud graders: exact strings, exact modes, exact owners. `make test-linux` runs them in the containers.

- Add a **local package repo exercise**: build a small RPM or DEB (`fpm`, `rpmbuild`) and install it from a local repo mirror. This showcases package management beyond `yum install`.

Environment limits to document honestly: SELinux enforcement and `firewalld` need a full OS, not a container. Cover them in an optional VM (Multipass, Vagrant or a local Rocky VM) with the same scripts, and note in the README which controls are container-testable and which are VM-only.

DoD: `make up && make test-linux` passes from a clean clone; running the hardening script twice produces no changes the second time.

Evidence: test output screenshot, a short write-up titled "Hardening a Linux server idempotently".

Extra step: SELinux and firewalld in a VM (closes the Phase 1 container limit)

Containers share the host kernel, so they cannot enforce SELinux or run `firewalld`. Add one small Rocky Linux VM for these two controls and keep everything else in containers.

1. Pick a VM tool

Option	Fit
Vagrant + VirtualBox (or libvirt)	Easiest: the Rocky box is one file in the repo and `vagrant up` rebuilds it
KVM/libvirt + Rocky cloud image + cloud-init	Lighter and more Linux-native, with more setup
Multipass	Ubuntu only, so it does not match a RHEL focus

Start with Vagrant. Use the libvirt provider if KVM is already running, since it conflicts with VirtualBox.

2. Define the VM in the repo

```
# linux/vm/Vagrantfile
Vagrant.configure("2") do |config|
  config.vm.box = "rockylinux/9"
  config.vm.hostname = "web-node-vm"
  config.vm.provider "virtualbox" do |vb|
    vb.memory = 2048
    vb.cpus = 2
  end
  config.vm.provision "shell", path: "../harden.sh"
  config.vm.provision "shell", path: "../vm-only/selinux.sh"
  config.vm.provision "shell", path: "../vm-only/firewall.sh"
end
```

Reusing the same `harden.sh` as the containers shows the Linux work is consistent across both environments.

3. Split the scripts by what each environment can test

```
linux/
├── harden.sh            # users, sshd, cron, ACLs, PAM  (container + VM)
├── vm-only/
│   ├── selinux.sh       # VM only
│   └── firewall.sh      # VM only
└── tests/
    ├── container/*.bats
    └── vm/*.bats
```

- **SELinux:** `setsebool -P httpd_can_network_connect on`, `semanage port -a -t ssh_port_t -p tcp 2222`, `restorecon -Rv /path`.
- **firewalld:** `firewall-cmd --permanent --add-service=http`, `firewall-cmd --permanent --add-port=8080/tcp`, then `firewall-cmd --reload`.
- Keep both scripts idempotent: check before changing, for example with `firewall-cmd --query-service=http`.

4. Write tests that check the real state

- **SELinux:** `getenforce` returns `Enforcing`, `ls -Z` shows the expected context, and `ausearch -m avc -ts recent` finds no denials.
- **firewalld:** `firewall-cmd --list-all` contains the expected services and ports; a `curl` from the host to the forwarded port succeeds, and a port that should be blocked times out.

- Run them with `vagrant ssh -c "sudo bats /vagrant/tests/vm"`.
- Demo idea: change a file's context on purpose so the service gets an AVC denial, diagnose it from the audit log, and fix it with `restorecon`.

5. Wire it into the Makefile and README

```
make test-linux      # containers only, fast, runs in CI
make test-linux-vm   # vagrant up + VM tests, run locally
```

Control	Container	VM
Users, sshd, cron, ACLs, PAM limits	yes	yes
SELinux enforcement	no	yes
firewalld rules	no	yes

6. Know the limits

- GitHub-hosted runners cannot easily run nested VMs, so the VM tests stay a local step. Capture a recording or screenshot of the passing run as evidence.
- Allow about 2 GB RAM for the VM, and run `vagrant halt` before starting minikube if memory is tight.
- Do not use privileged containers running systemd as a workaround: they do not give real SELinux enforcement.

Phase 2: Docker (from the Docker labs)

Goal: production-quality images and a multi-service stack.

Build

- Multi-stage Dockerfile for the Django app: small base, pinned versions, non-root user, `HEALTHCHECK`, `.dockerignore`, no secrets in layers.
- `docker-compose.yml` with app, MariaDB, reverse proxy; user-defined networks (front/back), named volumes, env files, `depends_on` with health conditions.
- Apply the **MariaDB datadir lesson**: do not rely on a pre-mounted `/var/lib/mysql`; make the data path explicit and documented, with an init script using `mysql_install_db` where required. Turn this into an ADR.
- Practise `docker cp`, `docker exec`, log inspection, network inspection, and image slimming (compare image size before and after, record it in the README).

- Add `hadolint` and `trivy image` scans as `make lint` and `make scan`.
- Run a **local registry** (`registry:2`) and push/pull images from it.

DoD: one command brings the stack up; the app survives a DB container restart with data intact; scan reports no critical findings (or documented exceptions).

Evidence: before/after image size table, scan report, Compose architecture diagram.

Phase 3: Kubernetes on minikube (from the Kubernetes labs)

Goal: run the same app on Kubernetes, covering both lab topics and the missing fundamentals.

Build (reinforces labs)

- Pod with two containers sharing an `emptyDir` (e.g. app writes logs, sidecar ships or tails them).
- Deployment with **rolling update** strategy; practise `kubectl rollout status|history|undo`.
- Explain the Deployment-to-ReplicaSet relationship with a short demo (delete a pod, watch self-healing).
- **Job** for Django migrations; **CronJob** for nightly DB backup written to a PVC.

Build (closes gaps)

- Namespaces (`strata-dev`, `strata-prod`), labels and selectors.
- **ConfigMap and Secret** for app config and DB credentials.
- **PersistentVolumeClaim** plus a **StatefulSet** for MariaDB.
- **Services** (ClusterIP) and **Ingress** (`minikube addons enable ingress`) with a local hostname via `/etc/hosts`.
- **Liveness, readiness and startup probes**; resource requests and limits.
- **HPA** (`minikube addons enable metrics-server`) with a simple load test (`hey` or `ab`).
- **RBAC:** a ServiceAccount with a least-privilege Role for the app.
- **NetworkPolicy:** restrict DB access to the app only (start minikube with `--cni=calico` so policies are enforced).
- Kustomize `base` and `overlays/local`.

DoD: `make k8s-up` deploys the whole stack; killing a pod causes no downtime; a bad image tag rolls back cleanly; the DB is unreachable from unrelated pods.

Evidence: manifests with comments, a rollout-and-rollback terminal recording, a NetworkPolicy before/after test.

Phase 4: CI/CD pipeline

Goal: every push is linted, tested, built, scanned and (locally) deployed.

Build

- GitHub Actions workflow: `lint` (shellcheck, hadolint, yamllint, `kubeconform`), `test` (app tests plus Linux bats tests), `build` (Docker Buildx), `scan` (Trivy), `deploy-local` job that spins up `kind` or minikube inside the runner and applies the manifests, then smoke-tests the Ingress.
- Use **`act`** to run the workflow locally during development so you are not waiting on remote runners.
- Tag-triggered release job: on `v*` tags, build and push versioned images (to the local registry for now; GHCR is a free, AWS-free option if you want a public artifact).
- Cache layers and dependencies; add a status badge to the README.

DoD: a PR cannot merge with a failing check (document the intended branch protection rule even if you apply it manually).

Evidence: green pipeline screenshot, badge, a deliberately broken PR showing the pipeline catching it.

Phase 5: Packaging and GitOps

Goal: show deployment maturity beyond `kubectl apply`.

Build

- Convert the manifests into a **Helm chart** with `values-local.yaml`; add `helm lint` and `helm template` to CI.
- Install **Argo CD** on minikube and point it at the repo's `k8s/overlays/local` path (or the chart). Make a change in Git and watch the cluster reconcile automatically; demonstrate drift correction by editing a resource by hand and letting Argo CD revert it.
- Optional: a local Git server (Gitea in Docker) so the full GitOps loop works fully offline.

DoD: Git is the single source of truth; a manual `kubectl edit` is reverted automatically.

Evidence: Argo CD sync screenshot, short "GitOps in 10 minutes" write-up.

Phase 6: Observability

Goal: see and reason about the system.

Build

- Install `kube-prometheus-stack` via Helm with trimmed resource settings suited to minikube (allocate roughly 4 CPUs and 6-8 GB RAM to the cluster).
- Expose Django metrics (`django-prometheus`), build one Grafana dashboard (request rate, error rate, latency, pod restarts), and add one alert rule (for example, pod crash looping or high 5xx rate).
- Centralised logs are optional (Loki + Promtail); a simpler alternative is a structured JSON logging standard documented in an ADR.

DoD: inducing a failure (scale DB to zero) visibly fires an alert and shows on the dashboard.

Evidence: dashboard screenshot, alert firing screenshot, the incident story.

Phase 7: IaC, docs and runbooks

Goal: codify the environment itself and make the project readable by a stranger.

Build

- **Terraform** with the `kreuzwerker/docker` and `hashicorp/kubernetes` providers (or `helm` provider) to create the Docker network, containers or namespaces locally. Structure it as modules with an `envs/local` environment so a future `envs/aws` just swaps providers.
- **Ansible** playbooks run against the Compose "servers" using SSH from the jump host, replacing the shell hardening script with a role (`roles/hardening`). Keep the shell version as a comparison, noting the trade-offs in an ADR.
- **Runbooks** derived from your lab gotchas, written as incident-style pages: "sshd won't restart after config change", "MariaDB won't start: datadir locked", "cron job not running: check cron.allow", "sudo context dropped mid-task".
- **ADRs** for key choices: Compose vs Kubernetes for each concern, Kustomize vs Helm, bash vs Ansible, container vs VM for SELinux.
- Write `docs/lab-mapping.md`, a table linking each KodeKloud lab category to the file(s) that implement it. This is your honest bridge between "completed labs" and "built project".

DoD: a new reader can go from clone to running stack using only the README.

Phase 8: AWS integration (deferred, design now)

Nothing here needs AWS access today. Prepare the seams so integration is additive:

Local component	AWS target	Preparation now
minikube	EKS	Keep manifests cluster-agnostic; Kustomize `overlays/aws` stub
`registry:2` / GHCR	ECR	Image repo and tag set through a variable, never hard-coded
MariaDB StatefulSet	RDS	App reads DB host, port, credentials only from env/Secret
PVC (hostPath)	EBS via CSI	Reference a `StorageClass` by name in values files
Local backup CronJob target	S3	Backup script takes a destination URI; local path today, `s3://` later
Docker "servers"	EC2	Terraform `modules/compute` interface with `envs/aws` stub
Docker networks	VPC, subnets, security groups	Model them as a `network` module with the same inputs
Secret objects	Secrets Manager / IAM roles	Keep secret access behind one config layer
Terraform local state	S3 backend with locking	Backend block isolated in `envs/*/backend.tf`

When AWS access returns, your earlier CLI lab work (key pairs, security groups, EBS snapshots, IAM, S3 sync) becomes the manual baseline you automate in Terraform, and the "passwordless SSH to EC2" flow becomes a Terraform plus Ansible pipeline that no longer risks an expiring lab session.

7. Suggested timeline

Week	Focus	Milestone
1	Phase 0 and 1	`v0.1.0`: hardened Linux containers with passing tests
2	Phase 2	`v0.2.0`: secure multi-service Compose stack with scans
3-4	Phase 3	`v0.3.0`: full app on minikube, rollback and NetworkPolicy demos
5	Phase 4	`v0.4.0`: CI pipeline green, badge in README
6	Phase 5 and 6	`v0.5.0`: Helm, Argo CD and a Grafana dashboard
7	Phase 7	`v1.0.0`: Terraform, Ansible, runbooks and ADRs complete
Later	Phase 8	`v2.0.0`: AWS overlay (EKS, ECR, RDS, S3)

Work one phase at a time and tag at the end of each; the tag history itself tells the story.

8. Local resource tips

- Start minikube with explicit limits: `minikube start --cpus=4 --memory=6144 --driver=docker --cni=calico`.
- Use `minikube image load` or the registry addon to avoid pushing to a remote registry.
- Stop heavy stacks (Prometheus, Argo CD) when not demonstrating; keep a `make k8s-down` that fully cleans up.
- Prefer Compose for Phases 1-2 so you are not paying cluster costs for what a container can show.

9. Making it portfolio-ready

1. **README first impression:** one-line pitch, architecture diagram, a 60-90 second demo GIF, a quickstart of three commands, and a CI badge.
2. **Show decisions, not only outputs:** ADRs and runbooks demonstrate judgement, which reviewers value more than a long tool list.

3. **Be honest about scope:** a "Limitations" section (local only, SELinux via VM, AWS deferred) builds credibility.
4. **Write 2-3 short posts or articles:** for example "Turning sysadmin labs into idempotent code", "Rollbacks and NetworkPolicies on minikube", "GitOps with Argo CD in a laptop cluster".
5. **Resume and LinkedIn bullets (adapt with your real results):**
   - Built a local-first platform on Docker and Kubernetes (minikube) with CI/CD, GitOps (Argo CD), and Prometheus/Grafana observability.
   - Automated Linux hardening (SSH, PAM, ACLs, cron, packages) with idempotent scripts and Ansible, verified by automated tests.
   - Designed provider-agnostic Terraform modules and Kustomize overlays to enable AWS migration (EKS, ECR, RDS, S3) without app changes.
6. **Pin the repo** on GitHub, add topics (`devops` , `kubernetes` , `docker` , `gitops` , `terraform`), and link it from your CV.

10. Quick-start checklist

- [ ] Create repo, add README, `.gitignore` , license; first commit with Conventional Commit message
- [ ] Install: Docker, minikube, kubectl, Helm, Kustomize, Terraform, Ansible, `pre-commit` , `act` , Trivy, hadolint, shellcheck, bats-core
- [ ] Phase 0 workflow in place and a `v0.1.0` tag planned
- [ ] Phase 1 containers building and first test passing
- [ ] `lab-mapping.md` started on day one and updated every phase

Suggested first step: do Phase 0 and the first half of Phase 1 in a single sitting: repo, hooks, one `web-node` container with sshd, and one failing-then-passing test. That gives you momentum and a visible first commit history.